



UNIP
UNIVERSIDADE PAULISTA

Roteiros

Programação Orientada a Objetos com C#

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Construindo um Programa de Console em C# 14, com .NET 10: Tipos, Nullabilidade, Pattern Matching, Records e Propriedades	ROTEIRO 1
---	---	-------------------------

ROTEIRO DE IMPLEMENTAÇÃO (PASSO A PASSO)

Etapa 0 – Preparação do projeto (ambiente)

1. Criar um **projeto de Console** no Visual Studio 2026, com o **.NET 10** como alvo (Target Framework).
2. Conferir se o projeto está compilando em **C# 14**. Se for preciso ajustar, configure a versão da linguagem no projeto (por exemplo, via propriedades do projeto ou editando o .csproj).
3. Verificar se a **análise de nulidade** está habilitada no projeto (para que string? faça sentido e gere avisos úteis).

Etapa 1 – Primeira saída no console (top-level statements)

1. No arquivo principal (por exemplo, Program.cs), escrever uma primeira linha usando Console.WriteLine para imprimir uma saudação.
2. Executar (F5 ou Ctrl+F5) e confirmar que a mensagem aparece.

Checkpoint: o console deve exibir uma frase simples de boas-vindas.

Etapa 2 – Tipos, inferência com var e interpolação de strings

1. Declarar uma variável inteira (por exemplo, um ano) com int.
2. Declarar uma variável de texto com var (o compilador infere string).
3. Imprimir uma frase usando **interpolação**: \$"...", inserindo as duas variáveis no texto.

Checkpoint: o console deve mostrar um texto com o nome e o ano.

Etapa 3 - Nullabilidade, ?? e ??=

1. Declarar uma variável string? inicializada com null.
2. Criar uma segunda variável (por exemplo, `exibir`) usando o operador `??`, definindo um texto padrão caso a primeira seja nula.
3. Imprimir `exibir`.
4. Usar o operador `??=` para atribuir um valor à variável string? apenas se ela ainda estiver nula.
5. Imprimir a variável novamente para verificar que agora possui um texto.

Checkpoint: uma das linhas deve imprimir o texto padrão (quando ainda era null), e outra linha deve imprimir o valor atribuído depois.

Etapa 4 - switch como expressão com pattern matching

1. Declarar uma variável do tipo `object?` recebendo um valor (por exemplo, um número inteiro).
2. Criar uma variável string chamada, por exemplo, `descricao`, cujo valor seja definido por um switch **como expressão**.
3. Incluir, no switch, pelo menos estes casos:
 - `null`
 - um padrão para `int` com uma condição `when` (por exemplo, `"maior que 100"`)
 - um padrão para `string`, usando o tamanho do texto
 - um caso "padrão" (`_`)
4. Imprimir `descricao`.

Checkpoint: ao usar um inteiro maior que 100, a descrição deve cair no caso correspondente.

Etapa 5 – Função local static e expressão de corpo (expression-bodied)

1. Criar uma **função local** (no mesmo arquivo), com retorno decimal, recebendo:
 - um preço (decimal)
 - uma taxa opcional com valor padrão (por exemplo, 0.12m)
2. Implementar a função com sintaxe de **expressão de corpo** (usando =>).
3. Chamar a função passando um preço e imprimir o resultado.

Checkpoint: o resultado deve refletir o preço somado da taxa (a formatação do decimal pode variar conforme a cultura do sistema).

Etapa 6 – record e with-expression (cópia com alteração)

1. Declarar um record chamado Produto, com três componentes (por exemplo: Id, Nome, Preço).
2. Criar uma instância p1 com valores iniciais.
3. Criar p2 a partir de p1 usando with, alterando apenas o preço.
4. Imprimir p2 no console.

Checkpoint: a impressão do record deve mostrar os campos do produto (o formato exato da impressão pode variar).

Etapa 7 – Classe com propriedade e o token contextual field (C# 14)

1. Declarar uma classe Config com uma propriedade Tema do tipo string.
2. Implementar o set da propriedade com validação:
 - se receber null, lançar ArgumentNullException
 - caso contrário, atribuir ao backing field usando o token contextual field
3. Criar um objeto Config, atribuir um tema válido e imprimir o valor.
4. (Teste extra recomendado) tentar atribuir null e observar a exceção.

Base conceitual do token field: ele permite acessar o backing field gerado pelo compilador dentro do accessor.

Etapa 8 – Classe Cliente e atribuição condicional com ?. no destino (C# 14)

1. Declarar uma classe Cliente com uma propriedade Total do tipo decimal.
2. Declarar uma variável Cliente? e inicializar com uma instância.
3. Usar uma linha com **atribuição composta** no formato:
 - cliente?.Total += ...;
4. Imprimir cliente?.Total.

Esse uso (com ?. no lado esquerdo da atribuição) faz parte da proposta de **null-conditional assignment** em C# 14.

Checkpoint: se cliente for null, a linha não deve lançar exceção; se houver instância, o total deve ser incrementado.

ENTREGÁVEIS

No relatório, incluir:

1. O resumo teórico (conforme orientações).
2. O código-fonte completo comentado (com os trechos que implementam todas as etapas).
3. Um recorte da saída do console (captura de tela ou transcrição curta).
4. As referências em ABNT.

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Desenvolvendo um Programa de Console em C#: As Condicionais e os Laços (com .NET 10 e C# 14)	ROTEIRO 2
---	---	-------------------------

ROTEIRO DE IMPLEMENTAÇÃO (PASSO A PASSO)

Etapa 0 – Preparação do projeto

1. Criar um **projeto de Console** no Visual Studio, escolhendo o **.NET 10** (ou o alvo indicado pelo professor).
2. Compilar e executar o projeto vazio para confirmar que o ambiente está funcional.
3. No arquivo principal, manter using System; no topo (se o modelo do projeto não o incluir automaticamente).

Verificação: o projeto deve compilar e abrir o console ao executar.

Etapa 1 – Mensagens iniciais no console

1. Imprimir um título (por exemplo: "demonstração de condicionais e laços").
2. Imprimir uma linha em branco para organizar a saída.

Verificação: o console deve mostrar um cabeçalho legível.

Etapa 2 – Identificadores case-sensitive

1. Declarar duas variáveis inteiras com nomes que diferem apenas por maiúsculas/minúsculas (por exemplo, aluno e Aluno).
2. Atribuir valores diferentes a cada uma (por exemplo, 10 e 20).
3. Imprimir os dois valores, deixando claro qual nome corresponde a qual valor.

Verificação: o console deve exibir dois números distintos, um para cada identificador.

Etapa 3 – Classificação por nota com if / else if / else

1. Declarar uma variável nota do tipo int.
2. Imprimir o valor da nota antes da classificação.
3. Implementar uma cadeia de condições com faixas, por exemplo:
 - abaixo de 50
 - abaixo de 70
 - abaixo de 90
 - caso contrário (a faixa final)
4. Em cada bloco, imprimir uma mensagem correspondente ao desempenho.

Testes obrigatórios: executar o programa várias vezes alterando a nota (por exemplo, 40, 60, 75 e 95) e registrar no relatório o que mudou.

Verificação: somente uma mensagem de resultado deve aparecer por execução.

Etapa 4 – Comando com switch

1. Declarar uma variável comando do tipo char.
2. Exibir no console qual comando foi escolhido.
3. Criar um switch com três casos (por exemplo, A, D, F) e um caso padrão (default).
4. Em cada caso, imprimir a ação correspondente e encerrar o caso com break.

Testes obrigatórios: executar com A, com D, com F e com um caractere diferente (por exemplo, X) para confirmar o comportamento do default.

Verificação: a saída deve mudar de acordo com o comando e deve haver uma mensagem específica no caso desconhecido.

Etapa 5 – Contagem com o laço for

1. Antes do laço, imprimir uma frase explicando o que será feito (por exemplo, "contar de 1 até 5").
2. Criar um for que inicialize um contador em 1, avance até 5 e incremente de 1 em 1.
3. Dentro do laço, imprimir o valor atual do contador a cada repetição.

Verificação: devem aparecer cinco linhas, uma para cada valor de 1 a 5.

Etapa 6 – Contagem regressiva com o laço while

1. Declarar uma variável contador (por exemplo, iniciando em 3).
2. Criar um while que execute enquanto o contador for maior que 0.
3. Dentro do laço:
 - imprimir o valor do contador;
 - decrementar o contador em 1.
4. Após o laço, imprimir uma mensagem de encerramento da contagem.

Testes obrigatórios: alterar o valor inicial (por exemplo, 5) e observar a diferença.

Verificação: a contagem deve parar no momento em que a condição deixar de ser verdadeira.

Etapa 7 – Encerramento com pausa do console

1. Imprimir uma mensagem pedindo para pressionar ENTER.
2. Ler uma linha do teclado para manter o console aberto.

Verificação: o programa deve esperar a tecla ENTER antes de finalizar.

EXTENSÕES (PARA A PARTE DE CRIATIVIDADE)

Escolher **ao menos uma**:

1. **Ler a nota do teclado** e validar a entrada (por exemplo, usando TryParse).
2. **Ler o comando do teclado**, aceitando letras minúsculas e convertendo para maiúsculas antes do switch.

3. **Transformar o programa em POO**, criando uma classe (por exemplo, DemonstradorDeFluxo) com métodos como:

- um método para mostrar o exemplo de case-sensitive;
- um método para classificar a nota;
- um método para processar o comando;
- um método para o for;
- um método para o while.

4. **Criar um menu com repetição**, usando um laço para permitir que o usuário execute cada parte (nota, comando, contagens) sem reiniciar o programa.

ENTREGÁVEIS

No relatório, incluir:

- o resumo teórico (conforme orientações);
- o código-fonte comentado;
- os testes realizados (quais valores foram usados e quais saídas foram observadas);
- as referências no padrão ABNT.

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Implementando um Minijogo de Batalha em Console com POO em C# (Classes e Objetos)	ROTEIRO 3
---	--	-------------------------

PASSO A PASSO DE IMPLEMENTAÇÃO (SUGESTÃO DE ORDEM)

Siga esta sequência para reduzir erros:

1. **Crie o projeto Console** e teste um "Olá, mundo" para validar o ambiente.
2. **Crie a classe Jogador** com as propriedades e o construtor.
3. Implemente ReceberDano no jogador e teste com um valor fixo (sem batalha ainda).
4. Implemente GanharPontos e teste somando pontos.
5. **Crie a classe Inimigo** com propriedades, construtor e ReceberDano.
6. Implemente Atacar em Jogador e em Inimigo:
 - comece com as mensagens e as validações;
 - depois, faça a chamada ao método ReceberDano do alvo.
7. **Crie a classe Jogo** e implemente Executar() até a parte de criar objetos e imprimir estados.
8. Implemente Batalhar(...) com o laço while e as pausas por ENTER.
9. Conecte tudo a partir de Main() e faça a execução completa.
10. Faça um ajuste final nos textos e nos comentários, mantendo o console legível.

CHECKLIST DE COMPILAÇÃO E EXECUÇÃO

Antes de entregar, verifique:

- o código compila sem erros;
- o Main() chama Executar() corretamente;
- o Batalhar(...) termina quando alguém chega à Vida == 0;
- a vida nunca fica negativa;
- o jogador ganha pontos somente quando derrota o inimigo;
- a segunda batalha só começa se o jogador estiver vivo.

EXTENSÕES (OPCIONAIS, PARA NOTA DE CRIATIVIDADE)

Escolha **uma ou mais** melhorias, mantendo o programa funcional:

- a adição de um terceiro inimigo;
- a criação de um "ataque crítico" com chance aleatória e dano maior;
- a inclusão de um item de cura com uso limitado;
- a alteração do ritmo do jogo (por exemplo, mostrar um resumo do turno antes do ENTER);
- a separação de responsabilidades (por exemplo, métodos menores para imprimir status).

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Implementando uma Conta Bancária com POO em C# (Encapsulamento, Propriedades e Métodos)	ROTEIRO 4
---	--	-------------------------

PASSO A PASSO DE IMPLEMENTAÇÃO

Passo 1 – Preparar o projeto

1. Crie um projeto **Console**.
2. Confirme que o projeto compila e executa com uma mensagem simples no console.

Passo 2 – Criar a classe e o campo

1. Crie um arquivo para a classe ContaBancaria.
2. Declare um campo saldo como private decimal.
3. Explique no relatório por que esse campo deve ser privado.

Passo 3 – Criar a propriedade Saldo

1. Crie uma propriedade pública Saldo.
2. Faça a leitura do saldo pelo get.
3. Restrinja a escrita externa usando private set (ou outra estratégia equivalente).
4. No relatório, explique por que o set não deve ser público.

Passo 4 – Criar o construtor com parâmetro opcional

1. Implemente um construtor que receba saldoInicial com valor padrão zero.
2. Inicialize o saldo com o valor recebido.
3. No relatório, descreva o que significa um parâmetro opcional e o sufixo m em decimal.

Passo 5 — Implementar Depositar

1. Crie o método retornando bool.
2. Aplique a regra: quantia deve ser maior que zero.
3. Em caso válido, some no saldo.
4. Retorne true ou false conforme o resultado.
5. No relatório, descreva a regra e por que ela existe.

Passo 6 — Implementar Sacar

1. Crie o método retornando bool.
2. Aplique as regras:
 - quantia deve ser maior que zero;
 - quantia não pode exceder o saldo.
3. Em caso válido, subtraia do saldo.
4. Retorne true ou false conforme o resultado.
5. No relatório, descreva as regras e a necessidade de evitar saldo negativo.

Passo 7 — Criar testes no Main

1. Instancie a conta com saldo inicial (por exemplo, 100m).
2. Exiba o saldo inicial.
3. Faça uma sequência de chamadas:
 - depósito válido;
 - depósito inválido;
 - saque válido;
 - saque inválido por exceder o saldo;
 - saque inválido por valor não positivo.
4. Após cada chamada, imprima:
 - o valor tentado;
 - se a operação foi aceita;
 - o saldo atual.

CHECKLIST DE VALIDAÇÃO

Antes de entregar, verifique:

- o código compila sem erros;
- o saldo não pode ser alterado diretamente de fora da classe;
- depósitos com valor não positivo são rejeitados;
- saques com valor não positivo são rejeitados;
- saques acima do saldo são rejeitados;
- o saldo é atualizado corretamente nos casos válidos.

EXTENSÕES (OPCIONAIS, PARA CRIATIVIDADE)

Escolha uma ou mais melhorias, mantendo a solução correta:

- a criação de um método Transferir(ContaBancaria destino, decimal quantia) usando Sacar e Depositar;
- a inclusão de um histórico simples de operações (lista de textos com data e tipo);
- a validação do saldo inicial para impedir valor negativo no construtor;
- a criação de um menu no console com opções "depositar", "sacar" e "sair".

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Abstração, Herança e Polimorfismo em C# (Aplicação de Console)	ROTEIRO 5
---	---	-------------------------

PARTE A - CONFIGURAÇÃO DO PROJETO NO VISUAL STUDIO

1. criar um novo projeto do tipo **Console App** (aplicação de console);
2. definir um nome para a solução e para o projeto (por exemplo, AnimaisP00);
3. confirmar que o projeto compila e executa (mesmo que ainda esteja com o template padrão).

PARTE B - IMPLEMENTAÇÃO PASSO A PASSO

1) Criar a classe abstrata Animal

Crie uma classe chamada Animal com estas características:

- ela deve ser **abstrata**;
- ela deve ter uma propriedade pública:
 - Nome, do tipo string;
- ela deve ter um **construtor público** que receba um string nome e atribua esse valor à propriedade Nome;
- ela deve declarar um método chamado EmitirSom com estas características:
 - ele deve ser **abstrato**;
 - ele deve retornar void;
 - ele não deve ter parâmetros.

Checklist do item 1

- a classe compila mesmo sendo abstrata;
- o método abstrato não tem corpo;
- o construtor inicializa a propriedade Nome.

2) Criar a subclasse concreta Cachorro

Crie uma classe Cachorro que:

- herde de Animal;
- tenha um construtor que receba string nome e repasse para o construtor da classe base;
- implemente EmitirSom com override, imprimindo no console uma mensagem com:
 - o nome do animal;
 - um som característico de cachorro.

Dica técnica

- use interpolação de strings com "\$"..." para imprimir o nome com a mensagem.

3) Criar a subclasse concreta Gato

Crie uma classe Gato que:

- herde de Animal;
- tenha um construtor que receba string nome e repasse para a classe base;
- implemente EmitirSom com override, imprimindo no console:
 - o nome do animal;
 - um som característico de gato.

4) Criar o programa de teste (classe Program e Main)

No ponto de entrada do programa:

1. declare **duas variáveis do tipo Animal**, mas instancie objetos concretos diferentes:
 - uma variável deve receber um Cachorro com um nome;
 - outra variável deve receber um Gato com um nome;
2. chame EmitirSom() nas duas variáveis.

O que você deve observar

- mesmo com o tipo declarado como Animal, o método executado deve ser o das subclasses (isso é o polimorfismo em ação).

PARTE C – COMPILAÇÃO, EXECUÇÃO E REGISTRO DO RESULTADO

1. compilar a solução (Build);
2. executar o programa (Run);
3. registrar no relatório:
 - a saída exibida no console (copie e cole como texto);
 - uma explicação curta de por que a saída mostra que houve polimorfismo.

PARTE D – EXTENSÕES (PARA CRIATIVIDADE E APRIMORAMENTOS)

Escolha **pelo menos uma** melhoria:

1. **Adicionar um terceiro animal**, como Vaca ou Pato, seguindo o mesmo padrão;
2. **Criar uma lista de animais** (List<Animal>), adicionar vários objetos e usar um laço para chamar EmitirSom() em cada um;
3. **Ler nomes do usuário** com Console.ReadLine() e criar os objetos com os nomes digitados;
4. **Validar entrada** (por exemplo, repetir a pergunta se o nome vier vazio).

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Interfaces e Polimorfismo em C# (Geometria em Aplicação de Console) Objetivo	ROTEIRO 6
---	--	-------------------------

PARTE A – CONFIGURAÇÃO DO PROJETO NO VISUAL STUDIO

1. criar um novo projeto do tipo **Console App** (aplicação de console);
2. definir o nome da solução e do projeto (por exemplo, GeometriaPOO);
3. executar o projeto inicial para confirmar que o ambiente compila e roda.

PARTE B – IMPLEMENTAÇÃO PASSO A PASSO

1) Criar a interface IForma

Crie uma interface chamada IForma com:

- um método CalcularArea() que retorne double;
- um método CalcularPerimetro() que retorne double.

Checklist do item 1

- a interface compila;
- os métodos têm apenas a assinatura (sem corpo).

2) Criar a classe Retangulo que implementa IForma

Crie uma classe concreta Retangulo com:

- a implementação de IForma;
- duas propriedades públicas do tipo double:
 - Largura;
 - Altura;
- um construtor que receba largura e altura e inicialize as propriedades;
- a implementação de:
 - CalcularArea() usando a fórmula do retângulo;
 - CalcularPerimetro() usando a fórmula do retângulo.

Ponto de atenção

- confirme no papel a fórmula de perímetro: ela deve considerar os dois pares de lados.

3) Criar a classe Circulo que implementa IForma

Crie uma classe concreta Circulo com:

- a implementação de IForma;
- uma propriedade pública do tipo double:
 - Raio;
- um construtor que receba raio e inicialize a propriedade;
- a implementação de:
 - CalcularArea() usando π e o raio ao quadrado;
 - CalcularPerimetro() (circunferência) usando π e o raio.

Dica técnica

- use Math.PI como valor de π .

4) Criar a classe do aplicativo (por exemplo, GeometriaApp) com Main

No método Main:

1. crie uma lista do tipo `List<IForma>`;
2. adicione pelo menos:
 - um retângulo, com valores numéricos definidos por você;
 - um círculo, com valor numérico definido por você;
3. percorra a lista com um laço (foreach) e para cada item:
 - imprima a área chamando `CalcularArea()`;
 - imprima o perímetro chamando `CalcularPerimetro()`;
 - imprima um separador visual (por exemplo, uma linha de traços).

O que você deve observar

- a variável do laço tem o tipo `IForma`, mas cada objeto pode ser de uma classe diferente;
- as chamadas de método devem executar o cálculo correspondente ao tipo concreto de cada forma.

PARTE C – COMPILAÇÃO, EXECUÇÃO E REGISTRO DO RESULTADO

1. compilar a solução;
2. executar o programa;
3. registrar no relatório:
 - o texto exibido no console (copiado e colado);
 - uma explicação curta mostrando onde aparece a interface e onde aparece o polimorfismo.

PARTE D - EXTENSÕES (PARA CRIATIVIDADE E APRIMORAMENTOS)

Escolha **pelo menos uma** melhoria:

1. **Adicionar uma terceira forma** (por exemplo, Triangulo), com as propriedades e as fórmulas corretas;
2. **Ler dados do usuário** com `Console.ReadLine()` e criar as formas a partir das entradas;
3. **Validar as entradas** (por exemplo, impedir valores negativos para largura, altura e raio);
4. **Formatar a saída** (por exemplo, limitar casas decimais com formatação numérica).

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Modelando Herança e Polimorfismo com as Classes Animal e Cachorro em C#	ROTEIRO 7
---	--	-------------------------

Roteiro prático (passo a passo para construir o programa)

Parte A – Preparando o projeto

1. No Visual Studio, criar **um projeto do tipo "Console App"** em C#.
2. Conferir se o projeto compila e executa com o programa inicial (o "Hello World" ou equivalente).
3. Localizar o arquivo principal (com Main), que pode estar em Program.cs (ou em arquivo equivalente, dependendo do template).

Observação: se o template estiver usando "top-level statements", você pode manter esse formato ou pode criar explicitamente uma classe Program com um método Main. As duas abordagens funcionam para esta atividade.

Parte B – Implementando a classe base Animal

4. Criar uma classe chamada **Animal**.
5. Dentro de Animal, criar uma **propriedade pública**:
 - o tipo deve ser string;
 - o nome deve ser Nome;
 - ela deve permitir **leitura e escrita** (isto é, com get; e set;).

6. Ainda dentro de `Animal`, criar um método público chamado `Comer`:
 - o retorno deve ser `void`;
 - o método deve escrever uma mensagem no console;
 - a mensagem deve incluir o valor de `Nome` (use interpolação de strings com `$"..."`).
7. Criar um método público chamado **FazerSom** na classe `Animal`:
 - o retorno deve ser `void`;
 - ele deve ser marcado como **virtual**;
 - ele deve escrever uma mensagem que represente um som genérico.

Checklist da Parte B

- a classe `Animal` existe;
- a propriedade `Nome` existe;
- o método `Comer()` imprime uma mensagem contendo `Nome`;
- o método virtual `FazerSom()` imprime um som genérico.

Parte C – Implementando a classe derivada `Cachorro`

8. Criar uma classe chamada **Cachorro** que **herde** de `Animal`.
9. Dentro de `Cachorro`, criar um método público chamado **AbanarRabo**:
 - o retorno deve ser `void`;
 - o método deve escrever uma mensagem no console;
 - a mensagem deve incluir `Nome`.
10. Na classe `Cachorro`, sobrescrever o método **FazerSom**:
 - você deve usar a palavra-chave **override**;
 - o método deve imprimir um som de cachorro (texto curto).

Checklist da Parte C

- a classe Cachorro herda de Animal;
- o método AbanarRabo() existe e usa Nome;
- o método FazerSom() foi sobrescrito com override.

Parte D – Testando no programa principal

11. No programa principal, criar um objeto do tipo **Cachorro**.

12. Atribuir um valor para a propriedade **Nome** do cachorro.

13. Chamar, nesta ordem:

- o método Comer(),
- o método AbanarRabo(),
- o método FazerSom().

14. Executar o programa e verificar se as mensagens aparecem como esperado.

Resultados esperados (em termos de comportamento)

- Comer() deve imprimir o nome do objeto.
- AbanarRabo() deve imprimir o nome do objeto.
- FazerSom() deve imprimir o som sobrescrito do cachorro (e não o som genérico).

Desafios de aprimoramento (para aumentar a nota)

Escolha **ao menos dois** itens abaixo:

1. **Criar a classe Gato** herdando de Animal, com:

- um método próprio (por exemplo, Arranhar()), e
- um override de FazerSom() com um som de gato.

2. Demonstrar polimorfismo com uma lista

- criar uma `List<Animal>` contendo um Cachorro e um Gato (ou outro animal);
- percorrer a lista com `foreach` e chamar `FazerSom()` em cada item;
- descrever, no relatório, por que o som muda conforme o tipo real do objeto.

3. Adicionar validações simples na propriedade Nome

- impedir `null` ou uma string vazia;
- registrar, no relatório, qual regra você aplicou e por quê.

4. Melhorar a legibilidade do console

- imprimir títulos, linhas em branco e separadores para deixar a saída mais clara.

  Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Orientada a Objetos com C# Título da Aula: Herança, Polimorfismo e Casting com Animal e Cachorro em C#	ROTEIRO 8
---	--	-------------------------

Roteiro de implementação (passo a passo)

Parte A – Preparando o ambiente e o projeto

1. Criar um projeto **Console App** em C#.
2. Executar o projeto vazio para confirmar que o ambiente está funcional.
3. Identificar onde ficará o código principal (arquivo com o ponto de entrada).

Parte B – Construindo a classe base **Animal**

4. Criar uma classe chamada **Animal**.
5. Adicionar uma **propriedade pública** chamada **Nome**:
 - tipo string;
 - com get; e set;.
6. Adicionar um método público chamado **Comer()**:
 - retorno void;
 - deve escrever uma frase no console contendo o Nome;
 - usar interpolação de string com \$"...".
7. Adicionar um método público chamado **FazerSom()**:
 - retorno void;
 - deve ser marcado com **virtual**;
 - deve escrever um texto de som genérico.

Verificação 1

- O projeto compila com a classe Animal criada.
- A classe contém Nome, Comer() e virtual FazerSom().

Parte C – Construindo a classe derivada Cachorro

8. Criar uma classe chamada **Cachorro** que herde de Animal.
9. Adicionar um método público chamado **AbanarRabo()**:
 - retorno void;
 - deve escrever uma frase no console contendo o Nome.

10. Sobrescrever o método FazerSom() em Cachorro:

- usar override;
- imprimir um som característico.

Verificação 2

- O projeto compila com as duas classes.
- Cachorro possui AbanarRabo() e override FazerSom().

Parte D – Criando animal1 e testando o comportamento da classe base

11. No programa principal, criar uma variável chamada **animal1** do tipo Animal, instanciando Animal.
12. Atribuir um valor para animal1.Nome.
13. Chamar animal1.FazerSom() e observar a saída.

Verificação 3

- A saída corresponde ao som genérico, pois o objeto é realmente um Animal.

Parte E – Criando animal2 como Animal, mas instanciando Cachorro (polimorfismo)

14. Criar uma variável chamada animal2 do tipo Animal, instanciando **Cachorro**.
15. Atribuir um valor para animal2.Nome.
16. Chamar animal2.FazerSom() e observar a saída.

Verificação 4

- A chamada deve executar o override de Cachorro, pois o tipo real do objeto é Cachorro.

Parte F = Entendendo o erro: por que animal2.AbanarRabo() não compila

17. Escrever (e comentar) uma linha chamando animal2.AbanarRabo().
18. Compilar e observar o erro de compilação.
19. Explicar no relatório:
 - a variável é do tipo Animal;
 - Animal não declara AbanarRabo();
 - o compilador valida pela informação do tipo da variável (tipo estático).

Parte G – Fazendo o casting para acessar o método específico

20. Substituir a chamada inválida por uma chamada que faça **casting explícito** para Cachorro:

- converter animal2 para Cachorro;
- chamar AbanarRabo() no resultado.

21. Executar e observar a saída.

22. No relatório, explicar:

- o cast funciona porque o objeto real foi criado como Cachorro;
- se o objeto real não fosse um Cachorro, ocorreria erro em tempo de execução.

Parte H – Chamando um método que existe na base

23. Chamar `animal2.Comer()` e observar que funciona sem cast.

24. Explicar no relatório:

- `Comer()` pertence à classe base, então está disponível na referência do tipo `Animal`.

Desafios de aprimoramento (para aumentar a nota)

Escolher ao menos dois:

1. Usar `is` para verificar o tipo antes do cast

- verificar se `animal2` é `Cachorro`;
- apenas então chamar `AbanarRabo()`.

2. Usar o operador `as` e checar `null`

- converter com `as`;
- chamar `AbanarRabo()` apenas se a conversão não resultar em `null`.

3. Adicionar uma nova classe derivada

- por exemplo, `Gato : Animal`;
- sobrescrever `FazerSom()` com um som diferente;
- instanciar como `Animal animal3 = new Gato();` e comparar saídas.

4. Demonstrar polimorfismo com coleção

- criar uma `List<Animal>`;
- inserir `Animal`, `Cachorro` e (se existir) `Gato`;
- percorrer com `foreach` chamando `FazerSom()`.